JavaScript Asynchronous Programming

1 How JavaScript Runs Code

JavaScript is a **single-threaded** programming language.

This means:

- JavaScript has **one execution path**
- JavaScript can do **one thing at a time**
- JavaScript executes code **in order**

It does not matter how fast the computer is — JavaScript itself follows this rule.

2 Synchronous Execution

What does synchronous mean?

Synchronous code is executed **line by line**.
Each line must finish before the next line starts.

Example

```javascript
console.log("First");
console.log("Second");
console.log("Third");
```

Output

```
First
Second
Third
```

Explanation

- JavaScript executes the first line
- Then the second

- Then the third
- No line runs before the previous one finishes

This behavior is called **blocking execution**.

3 Why Synchronous Code Is a Problem

Synchronous execution is simple, but it has a serious limitation.

Some operations take **time**, such as:

- heavy calculations
- loading data
- waiting for servers
- reading files

If JavaScript waits synchronously for these operations, the entire application becomes unusable.

Blocking Example

```javascript
console.log("Start");

for (let i = 0; i < 5_000_000_000; i++) {
  // long-running task
}

console.log("End");
```

What happens?

- The browser freezes
- The page stops responding
- Buttons and scrolling stop working
- `"End"` is printed only after a long delay

This happens because JavaScript is **busy**.

4 The Core Limitation

JavaScript has:

- one call stack
- one thread
- one active task

So JavaScript **cannot wait** without blocking everything else.

This leads to a key question:

How can JavaScript handle slow tasks
without freezing the application?

5 Asynchronous Execution

What does asynchronous mean?

Asynchronous execution allows JavaScript to:

- start a task
- continue executing other code
- return to the task when it finishes

Important:

- Asynchronous does **not** mean multi-threaded
- JavaScript is still single-threaded

6 First Asynchronous Example: `setTimeout`

```javascript
console.log(1);

setTimeout(() => {
  console.log(2);
}, 1000);

console.log(3);
```

Output

```
None
1
3
2
```

Explanation

1. `console.log(1)` runs immediately
2. `setTimeout` schedules a task for later
3. JavaScript does not wait
4. `console.log(3)` runs immediately
5. After 1 second, the callback runs

JavaScript **delegates** the waiting.

7 JavaScript Does Not Work Alone

JavaScript runs inside an **environment**:

- in the browser
- or in Node.js

This environment provides features JavaScript does not have by itself.

In the browser, these features are called **Web APIs**.

8 What Are Web APIs?

Web APIs are tools provided by the browser that handle:

- timers
- network requests
- user events
- browser capabilities

They are **not part of JavaScript itself**.

Common Web APIs

- `setTimeout`
- `fetch`
- DOM events (`click`, `scroll`)
- Local Storage
- Geolocation

9 The Call Stack

The **Call Stack** is where JavaScript executes code.

Rules:

- Only one function runs at a time
- Functions are pushed onto the stack
- Functions are removed after execution

If the call stack is busy, nothing else can run.

10 The Callback Queue

When a Web API finishes its work:

- the callback function is placed in a **queue**
- the callback waits for JavaScript to be free

This waiting area is called the **Callback Queue**.

11 The Event Loop

The **Event Loop** is the mechanism that coordinates everything.

Its job is simple:

> When the call stack is empty,
> move the next callback from the queue into execution.

Event Loop Example

```javascript
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

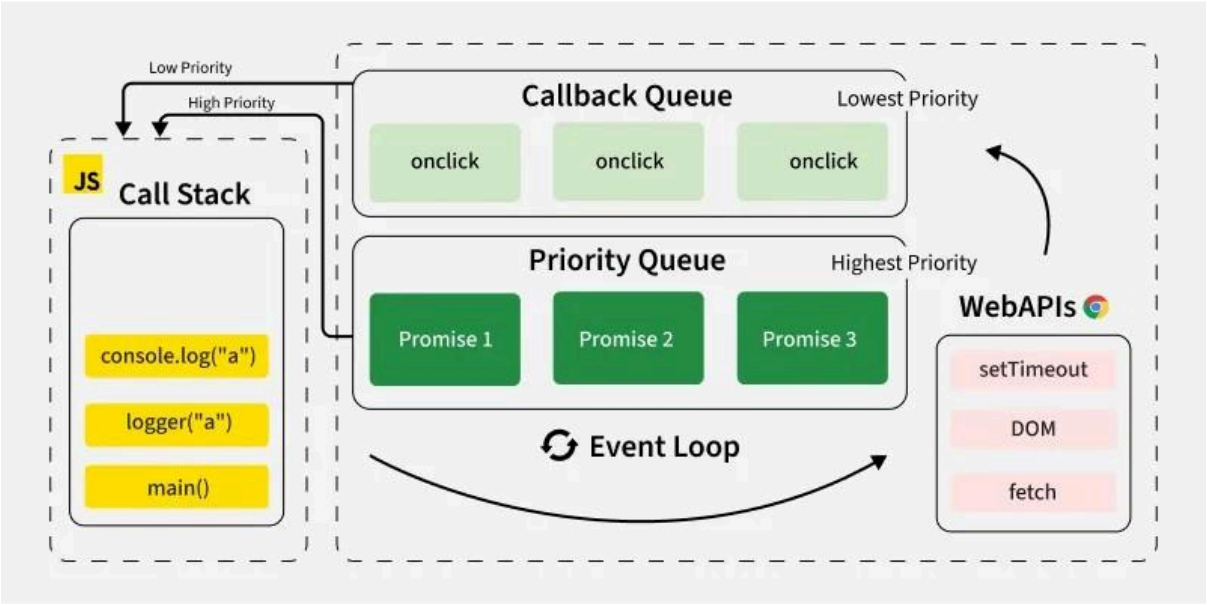
Output

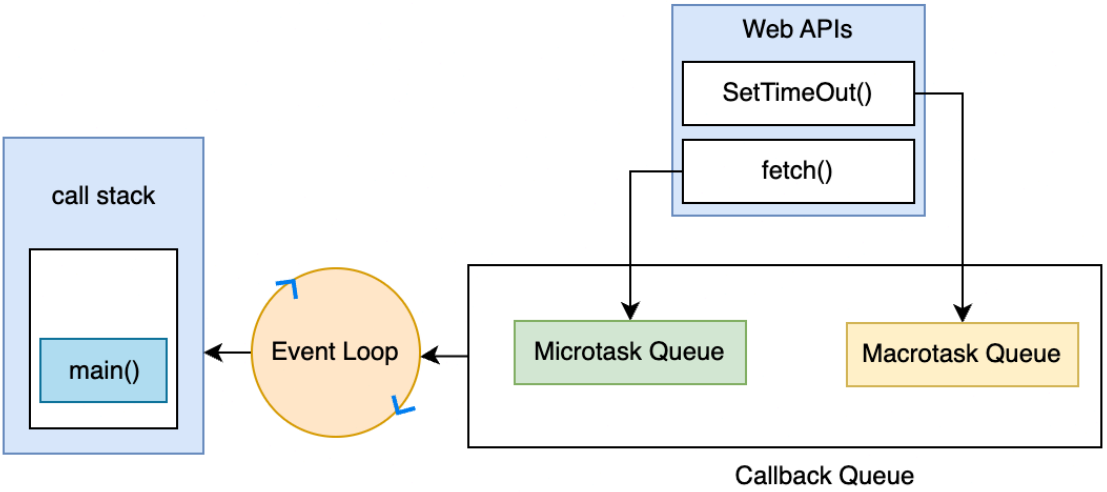
```none
A
C
B
```

Step-by-Step

1. `"A"` executes
2. `setTimeout` is handled by the browser
3. `"C"` executes
4. Call stack becomes empty
5. Callback moves to the stack
6. `"B"` executes

Even with `0ms`, async code runs **after** synchronous code.


Low Priority
High Priority
JS
Call Stack
console.log("a")
logger("a")
main()
Callback Queue
Lowest Priority
onclick
onclick
onclick
Priority Queue
Highest Priority
Promise 1
Promise 2
Promise 3
Event Loop
WebAPIs
setTimeout
DOM
fetch


Web APIs
SetTimeOut()
fetch()
call stack
main()
Event Loop
Microtask Queue
Macrotask Queue
Callback Queue

12 Why APIs Are Asynchronous

Most applications need data from servers:

- users
- posts
- products
- authentication

This data lives on **another computer**.

JavaScript sends a request and waits for a response.

But:

- the network takes time
- the server response time is unknown

So API communication **must be asynchronous**.

13 What Is an API?

An **API (Application Programming Interface)** is:

> A way for one program to request data or actions from another program.

In web development:

- JavaScript sends a **request**
- The server sends a **response**

JavaScript does not control when the response arrives.

14 The API Problem

Consider this code:

```javascript
const users = getUsersFromServer();
console.log(users);
```

This does **not** work.

Why?

- The data is not ready yet
- JavaScript already moved on

This is the same problem we saw with `setTimeout`.

15 JavaScript's First Solution: Callbacks

To solve this problem, JavaScript introduced **callbacks**.

What Is a Callback?

A **callback** is:

> A function passed to another function
> to be executed later, after an async task finishes.

In simple terms:

> "Call this function when you're done."

16 Callback Example

```javascript
function getData(callback) {
  setTimeout(() => {
    callback("Hello");
  }, 1000);
}

getData((result) => {
  console.log(result);
});
```

Explanation

- `getData` starts an async task
- JavaScript does not wait
- When the task finishes, the callback runs
- The result is handled **inside the callback**

17 Important Rule About Callbacks

> Asynchronous data exists **only inside the callback**.

This will **never work**:

```javascript
const data = getData();
console.log(data);
```

This **will work**:

```javascript
getData((data) => {
  console.log(data);
});
```

18 Callbacks and APIs

Before promises existed, APIs were handled using callbacks.

Example:

```javascript
function getUsers(callback) {
  setTimeout(() => {
    callback(["Ahmad", "Sara", "Lina"]);
  }, 1000);
}

getUsers((users) => {
  console.log(users);
});
```

The data arrives later, and the callback handles it.

19 The Problem with Callbacks

Callbacks work — but they don't scale well.

When multiple async operations depend on each other, callbacks become deeply nested.

Callback Nesting

```javascript
login((user) => {
  getProfile(user.id, (profile) => {
    getPosts(profile.id, (posts) => {
      getComments(posts[0].id, (comments) => {
        console.log(comments);
      });
    });
  });
});
```

This pattern is called **Callback Hell**.

20 Why Callback Hell Is a Problem

- Code becomes hard to read
- Debugging is difficult
- Error handling is messy
- Logic is unclear

Callbacks solved the async problem, but created a new one.

21 Why Callback Hell Forced a New Solution

By now, we understand that callbacks solved an important problem:

- they allowed JavaScript to handle asynchronous results

However, when applications became more complex, callbacks created new issues:

- deeply nested code
- poor readability
- difficult error handling
- hard maintenance

This problem is known as **Callback Hell**.

So JavaScript needed:

- a cleaner structure
- better error handling
- a more readable way to work with async code

This led to the creation of **Promise**

22 What Is a Promise?

A **Promise** is an object that represents a value that:

- does not exist yet
- will exist in the future
- may succeed or fail

In simple terms:

> A promise is a guarantee that
> “I will give you a result later.”

23 Promise States

Every promise is always in **one** of these states:

State	Meaning
Pending	The async operation is still running
Fulfilled	The operation finished successfully
Rejected	The operation failed

Once a promise is fulfilled or rejected, it is **settled** and cannot change again.

24 First Promise Example (Simple)

```javascript
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data loaded");
  }, 1000);
});
```

Explanation

- `new Promise` creates a promise
- `resolve` is called when the operation succeeds
- `reject` is called when the operation fails
- The promise starts in the **pending** state

25 Consuming a Promise with `.then()`

To use the result of a promise, we attach a `.then()` method.

```javascript
promise.then((result) => {
  console.log(result);
});
```

What `.then()` does

- It runs **after** the promise is fulfilled
- It receives the resolved value
- It replaces the need for a callback

26 Handling Errors with `.catch()`

Promises provide a clean way to handle errors.

```javascript
promise
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.error(error);
  });
```

Why this is better than callbacks

- Errors are handled in one place
- No deeply nested error checks
- Clear success vs failure logic

27 Rewriting a Callback Example Using Promises

Callback version (Callback Hell style)

```javascript
getUser((user) => {
  getProfile(user.id, (profile) => {
    getPosts(profile.id, (posts) => {
      console.log(posts);
    });
  });
});
```

Promise version (Flat & Readable)

```javascript
getUser()
  .then((user) => getProfile(user.id))
  .then((profile) => getPosts(profile.id))
  .then((posts) => {
    console.log(posts);
  })
  .catch((error) => {
    console.error(error);
  });
```

Why This Is Better

- Code flows from top to bottom
- No deep nesting
- Easy to read
- Centralized error handling

This technique is called **Promise Chaining**.

28 Promises and APIs (Very Important Connection)

Modern APIs in JavaScript return promises.

The most common example is `fetch`.

```javascript
fetch("https://api.example.com/users")
  .then((response) => response.json())
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.error(error);
  });
```

What happens here?

- `fetch` starts a network request

- It returns a promise immediately
- The response arrives later
- `.then()` runs when data is ready

This is why promises are essential for API work.

29 Important Rule About Promises

`.then()` does NOT run immediately
It runs **only when the promise is fulfilled**

Promises still use:

- Web APIs
- Event Loop

They just provide a **cleaner interface**.

30 Promises Do NOT Make Code Synchronous

Even though promise code looks clean, it is still asynchronous.

```javascript
console.log("Start");

fetch("https://api.example.com/users")
  .then(() => console.log("Fetched"));

console.log("End");
```

Output

```none
Start
End
Fetched
```

The Event Loop rules still apply.